

FarmerChat SDK

Drop an AI farm advisor into an existing app. Farmers ask about their crops and livestock by voice, photo or text, in their own language — and the host app writes three lines of code.

ANDROID SIZE	PLATFORMS	SCREENS	ENDPOINTS	CONFIG KNOBS
2.73 MB — core + views	4 8 packages	17 full journey	34 + token refresh	46 all optional

01 The problem

Smallholder farmers need timely, local, plain-language advice — which crop variety, when to sow, what the spots on those leaves mean. That expertise exists, but it doesn't reach them.

Partner organisations already have apps farmers use and trust: feed calculators, procurement tools, cooperative portals. Rebuilding an AI advisor inside each one is unaffordable. The SDK makes it an afternoon's work instead.

02 What it does

- **Ask any way**

Type a question, speak it, or photograph a diseased plant. Voice is transcribed server-side; photos go to image analysis.

- **Their language**

Language chosen on first run and switchable any time. Every string is fetched from the server, so copy updates without an app release.

- **Answers that continue**

Every reply carries follow-up questions, so a farmer can dig deeper without knowing what to ask next. Answers can be read aloud.

- **Advice that finds them**

A daily feed of location-relevant cards — crop guidance, seasonal tips, local weather — each opening straight into a conversation.

- **Nothing is lost**

Past conversations are saved and searchable, paged as the farmer scrolls back. Answers can be shared or saved as an image.

- **Looks like your app**

Brand colours, bubble styling, FAB, corner radii, fonts and copy overrides — the assistant reads as part of the host, not bolted on.

03 Integrating it

Android shown; the other platforms follow the same three steps with their own idioms.

1 Add the dependency

```
// core arrives transitively
implementation("org.digitalgreen.farmerchat:farmerchat-android-views:1.0.0")
```

2 Initialize once, at app start

```
FarmerChat.initialize(context, FarmerChatConfig
    .builder(FarmerChatEnvironment.PROD)
    .geoApiKey(BuildConfig.FC_GEO_API_KEY)
    .fabBackgroundColor(brandGreen)
    .build())
```

3 Give it an entry point

```
<!-- a floating button, anywhere in your layout -->
<org.digitalgreen.farmerchat.sdk.views.FarmerChatFab ... />

// or call it yourself
FarmerChat.launch(context)           // full journey
FarmerChat.openChat(context, question = "...") // straight to chat
```

What the host must provide

- A Google Geolocation key. Powers the location-aware advice feed and local weather.
- minSdk 26 and JVM target 17. Lower-minSdk hosts can keep 24 via `tools:overrideLibrary` plus a runtime API check.
- Play-listing permissions. The merged manifest gains microphone, camera and location.

04 Two ways to embed

MODE	THE FARMER GETS	SUITS
FULL_JOURNEY	Language choice, a daily advice feed, chat, history, settings and help — the whole assistant.	Apps making FarmerChat a headline feature.
CHAT_ONLY	Straight into a conversation. History and language sit in the chat's own app bar; no dashboard, no side menu.	Apps adding a quiet "ask an expert" affordance.

Either mode can also be embedded inline as a view inside an existing screen, rather than opened full-screen.

05 Platform coverage

PLATFORM	PACKAGES	REQUIRES
Android	core + Views <i>or</i> Compose	minSdk 26, JVM 17
iOS	Core + SwiftUI <i>or</i> UIKit	iOS 15+ (SwiftUI 16+)
React Native	single npm package	RN 0.76+ / Expo 52+
Web	single npm package	React 18

All four share one behavioural spec — the same endpoints, retry and timeout rules, token refresh, and screen flow — so a fix on one platform is expected on all four.

Figures measured from the current build: Android AARs from the published artifacts, screens and endpoints from the SDK specification.